

Ejercicio 1.(25 puntos)

Un proceso será el encargado de completar los valores de un arreglo de enteros de m posiciones con valores específicos. Para ello va a crear n procesos hijos, quienes cuentan con la función `int generarValor(int indice)`, que toma como argumento el índice del arreglo que queremos completar, y devuelve un número entero. Esta función sólo puede ser ejecutada en el contexto de procesos hijos. Por último, cuando el padre tenga el arreglo de m posiciones completo y sus hijos hayan terminado, deberá informar su valor utilizando la función `printArray(int[] array)`. Se pide:

- Escribir el pseudocódigo de un programa que cumpla con lo solicitado, utilizando pipes como medio de comunicación entre los procesos, asumiendo que m es múltiplo de n y que los hijos trabajarán de forma equitativa.
- Supongamos que `generarValor()` puede demorar un tiempo variable en cada caso, por lo que algunos procesos hijos pueden terminar su trabajo antes que otros, y queremos que no haya procesos hijos ociosos en caso de que aún queden items del arreglo sin procesar. ¿La solución provista en el inciso anterior funciona? En caso afirmativo, justificar. En caso negativo, explicar por qué y proponer una idea de solución.

```
(a) # define FORM(i, m) for (int i = 0; i < m; i++)
int generarValor (int indice); PRINT ARRAY (INT[] ARRAY,
int m, m);
int ARR[m];
int pipes[m][2];

int main () {
    // m = n * k, con n procesos hijos, n = |ARRAY|
    int k = m/n.
    FORM(i, m) pipe (pipes[i]);
    FORM(i, m) {
        pid_t pid = fork();
        if (pid == 0) TRABAJO INFANTIL (i, n, k);
    }
    FORM(i, m) close (pipes[i][WRITE]);
    FORM(i, m) {
        FORM(j, k)
            READ (pipes[i][READ], &ARR[i * k + j], sizeof(int));
    }
    FORM(i, m) WAIT (NULL);
    FORM(i, m) close (pipes[i][READ]);
    PRINT ARRAY (ARR);
    return EXIT_SUCCESS;
}
```

```

void trabajo infantil (int id, int m, int k) {
    int cur = id * m;
    int val;
    form(i, m) {
        close (pipes[i][read]);
        if (pipes[i][write] != pipes[id][write])
            close (pipes[i][write]);
    }
    form (i, k) {
        val = generar valor (cur);
        write (pipes[id][write], &val, sizeof(val));
        cur++;
    }
    close (pipes[id][write]);
    exit (EXIT_SUCCESS);
}

```

- (b) NO FUNCIONA, SI UN PROCESO HIJO TERMINA ANTES es porque escribió los k datos pedidos para cada índice correspondiente, cerró su extremo de pipe e hizo exit(),
-) si con que no estén ociosos se refieren a que estén vivos → en mi código los hijos se matan cuando hacen su trabajo. → no están ociosos, están muertos
 -) si se refieren a que ayuden a los lentos → PROPORCIONARÍA MAPA DE BITS / ARREGLO DE BOOLS pero REQUERIRÍA SINCRONIZACIÓN → OUT OF SCOPE.

Ejercicio 2.(25 puntos)

Tenemos un sistema donde existen dos tipos de procesos:

- **Proceso Tipo 1:** Recibe una solicitud de un usuario y agrega un trabajo a una cola de tareas. Esta cola tiene un tamaño máximo N , ya que no se pueden tener demasiadas tareas esperando en la memoria.
- **Proceso Tipo 2:** Toma trabajos de la cola (usando `pop()`) y realiza un trabajo de procesamiento.

Si la cola se llena, no se le deben agregar nuevos trabajos. La cola de trabajos no es atómica. La interacción de cada proceso con la cola se implementó hace poco y su comportamiento se puede representar con el siguiente pseudocódigo:

```
// Variables compartidas:  
mutex = sem_init(1)  
jobs = sem_init(0)  
buffer cola(N) // buffer de tamaño N
```

```
// Proceso Tipo 1  
trabajo = esperarPromptUsuario();  
mutex.wait();  
cola.add(trabajo);  
mutex.signal();  
jobs.signal();
```

```
// Proceso Tipo 2  
mutex.wait();  
jobs.wait();  
trabajo = cola.pop();  
mutex.signal();  
enviarResultado(trabajo);
```

Sin embargo, se reportó que esta parte del código no está funcionando correctamente y que no cumple exactamente con lo pedido. Nos encargaron la tarea de identificar qué está ocurriendo. Se pide:

- a) Encontrar qué problema/s tiene este código con respecto a la sincronización de los procesos. Justificar detalladamente la/s causa/s y corregirlo, cumpliendo con la consigna del problema.

(a) el problema en el código del proceso 2, donde existe un claro riesgo de deadlock ya que hace `mutex.wait()` y `jobs.wait()` tu proceso 1 hace `mutex.wait()` y queda bloqueado esperando un signal que jamás llega ya que a demás proceso 2 espera el `jobs.signal` del proceso 1 (que está bloqueado)

g) VARIABLES compartidas: `mutex = sem_init(1)`, `jobs = sem_init(0)`,
`buffer cola(N)`, `limite = sem_init(1)`

```
// proceso 1  
trabajo = esperarPromptUsuario();  
mutex.wait();  
limite.wait();  
cola.add(trabajo);  
mutex.signal();  
jobs.signal();
```

```
// proceso 2  
jobs.wait();  
mutex.wait();  
trabajo = cola.pop();  
limite.signal();  
mutex.signal();  
enviarResultado(trabajo);
```

ADemás, también estaba el error de que no estaba implementado el límite del buffer al momento de agregar un trabajo. Use un semaphore(N)

✖

b) Ahora se desea que k procesos Tipo 2 (siendo k mucho menor que N) trabajen concurrentemente, pero sólo puedan enviar un resultado cuando k procesos hayan extraído una tarea de la cola. Es decir, ningún proceso puede enviar el resultado inmediatamente después de retirar el trabajo de la cola, debe esperar a los otros procesos. Extender la solución del inciso anterior con esta restricción, asegurando que no haya problemas de concurrencia. Justificar la solución.

```
# define FORM(i, m) for (int i = 0; i < m; i++)
```

```
VARIABLES: mutex = sem_init(1)    int k
COMPROMISOS: jobs = sem_init(0)    int count = 0;
            buffer = cola(N)        barrier = sem_init(0)
            limite = sem_init(N)
```

// proceso 1

```
trabajo = esperarPromptUsuario();
mutex.wait();
limite.wait();
cola.add(trabajo);
mutex.signal();
jobs.signal();
```

// proceso 2

```
jobs.wait();
mutex.wait();
trabajo = cola.pop();
limite.signal();
count++;
if (count == k)
{ FORM(i, k) barrier.signal(); }
mutex.signal();
barrier.wait();
enviarResultado(trabajo);
```

Basicamente el error esta corregido, y ahora extendo el critical code area tq llevo un contador que hace C/proceso que saca un trabajo de la cola, entonces cuando el k -esimo proceso saca un trabajo enviara signal k -veces tq todos podran enviar su resultado de regreso.